Лабораторная работа №2
Визуализация и анализ производительности с использованием Grafana

Цель работы

– понимать назначение визуализации метрик в высоконагруженных системах;
– познакомиться с Grafana и её местом в стеке мониторинга;
– подключить Prometheus как источник данных Grafana;
– создавать панели (panels) и дашборды (dashboards);
– использовать запросы PromQL в Grafana;
– визуализировать RPS, latency, ошибки, CPU и память;
– анализировать изменение показателей во времени;
– сопоставлять нагрузку, производительность и использование ресурсов;
– выявлять признаки деградации системы по графикам;
– формулировать гипотезу о причине снижения производительности.

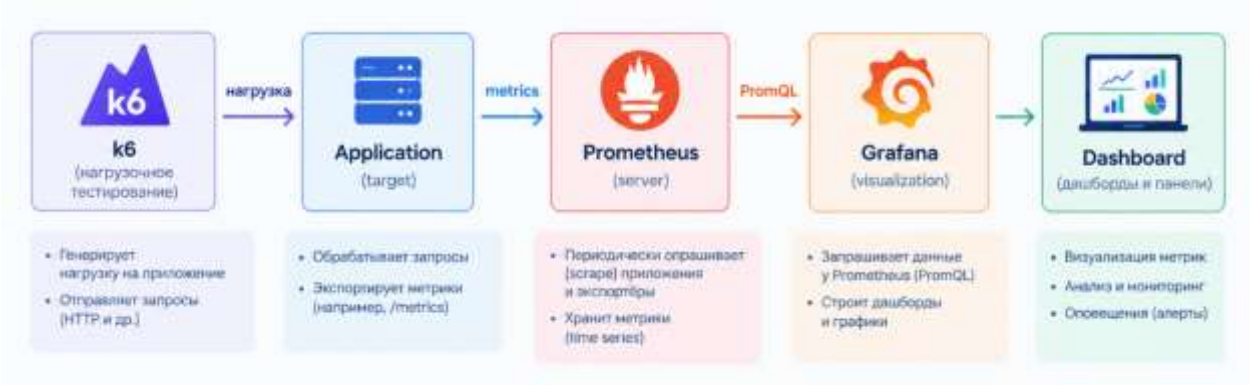
Grafana – платформа для визуализации и анализа данных мониторинга. Она не хранит данные, а подключается к источникам (Data Sources) и отображает их в виде графиков, таблиц, gauge и других панелей.

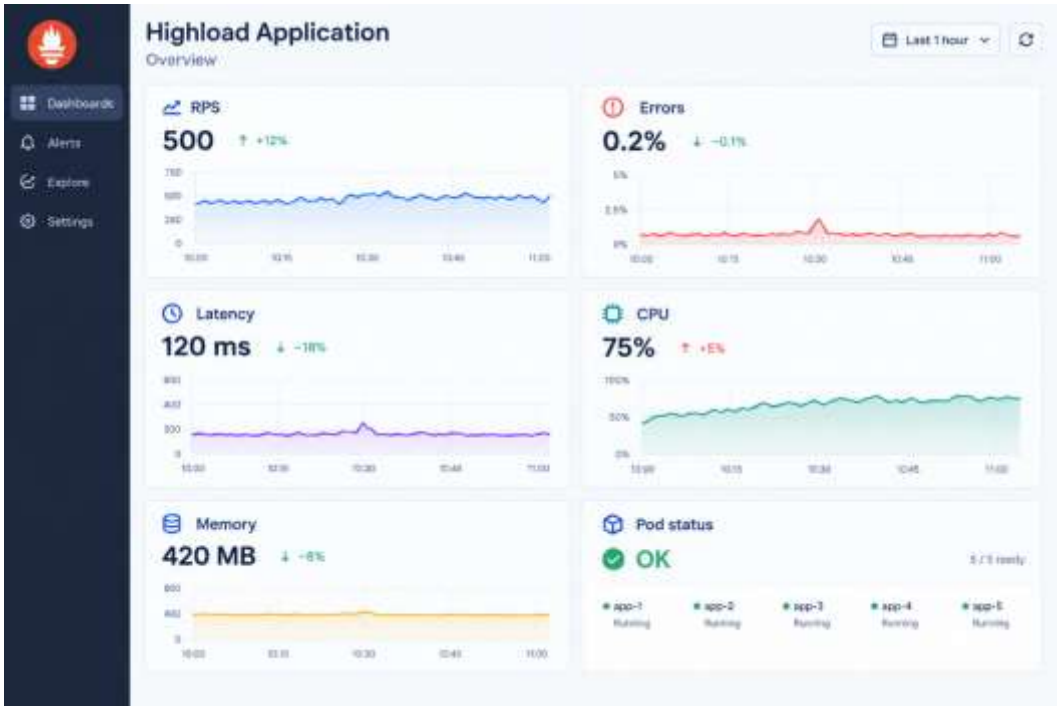
Grafana позволяет:

– подключать источники данных;
– выполнять запросы к данным;
– строить графики;
– создавать dashboards;
– объединять несколько показателей на одном экране;
– анализировать изменение метрик во времени;
– настраивать алерты по пороговым значениям.

Место Grafana в стеке мониторинга

– k6 создаёт нагрузку и показывает внешний результат.
– Prometheus собирает внутренние показатели.
– Grafana позволяет увидеть взаимосвязь этих показателей во времени.





Dashboard – набор панелей на одном экране:

Panel – отдельный элемент dashboard.

Типы панелей:

Тип	***Назначение***	***Пример***
Time series	Изменение во времени	RPS, latency, CPU
Stat	Текущее значение	RPS сейчас
Gauge	Показатель с порогами	CPU %
Table	Табличное представление	Список Pod'ов
Heatmap	Распределение значений	Latency по бакетам

Шаг 1. Проверка Grafana

Grafana устанавливается вместе с kube-prometheus-stack (ЛР2). Проверим

```
C:\Users\Anna>kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTARTS   AGE
alertmanager-prometheus-kube-prometheus-alertmanager-0   2/2     Running   0          8m14s
prometheus-grafana-5f577b8d79-8s94b                      3/3     Running   0          8m28s
prometheus-kube-prometheus-operator-8dd9d5fbb-znp2g      1/1     Running   0          8m28s
prometheus-kube-state-metrics-6bfc9c7c48-fk6q2           1/1     Running   0          45m
prometheus-prometheus-kube-prometheus-prometheus-0       2/2     Running   0          8m14s
prometheus-prometheus-node-exporter-965h5                1/1     Running   0          45m

C:\Users\Anna>kubectl get services -n monitoring
NAME                                      TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                      AGE
alertmanager-operated                     ClusterIP   None            <none>        9093/TCP,9094/TCP,9094/UDP   8m30s
prometheus-grafana                        ClusterIP   10.96.1.224     <none>        80/TCP                       46m
prometheus-kube-prometheus-alertmanager   ClusterIP   10.96.165.41    <none>        9093/TCP,8080/TCP            46m
prometheus-kube-prometheus-operator       ClusterIP   10.96.210.136   <none>        443/TCP                      46m
prometheus-kube-prometheus-prometheus     ClusterIP   10.96.234.242   <none>        9090/TCP,8080/TCP            46m
prometheus-kube-state-metrics             ClusterIP   10.96.155.20    <none>        8080/TCP                     46m
prometheus-operated                       ClusterIP   None            <none>        9090/TCP                     8m28s
prometheus-prometheus-node-exporter       ClusterIP   10.96.136.73    <none>        9100/TCP                     46m
```

Шаг 2. Доступ к Web-интерфейсу

```
C:\Users\Anna>kubectl port-forward -n monitoring service/prometheus-grafana 3000:80
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
```

Шаг 3. Вход в Grafana



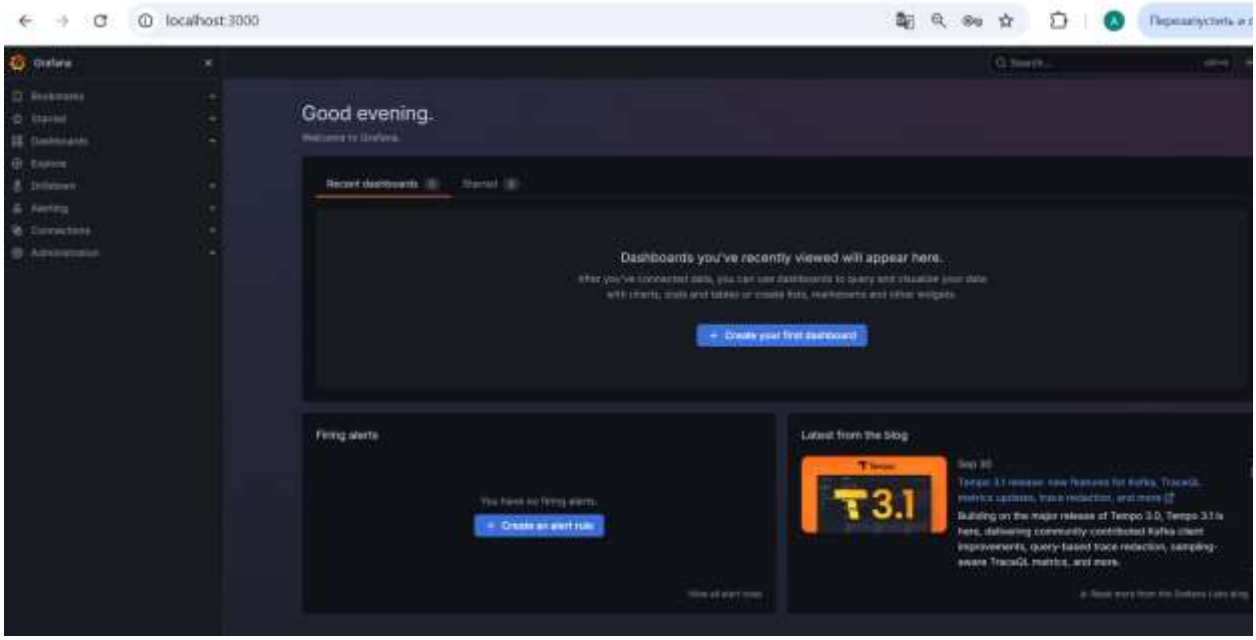
Логин по умолчанию: admin. Пароль – полученный из Secret.

Для Windows:

kubectl get secret -n monitoring prometheus-grafana -o jsonpath="{.data.admin-password}" | powershell -Command "$input | %{ [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($_)) }"

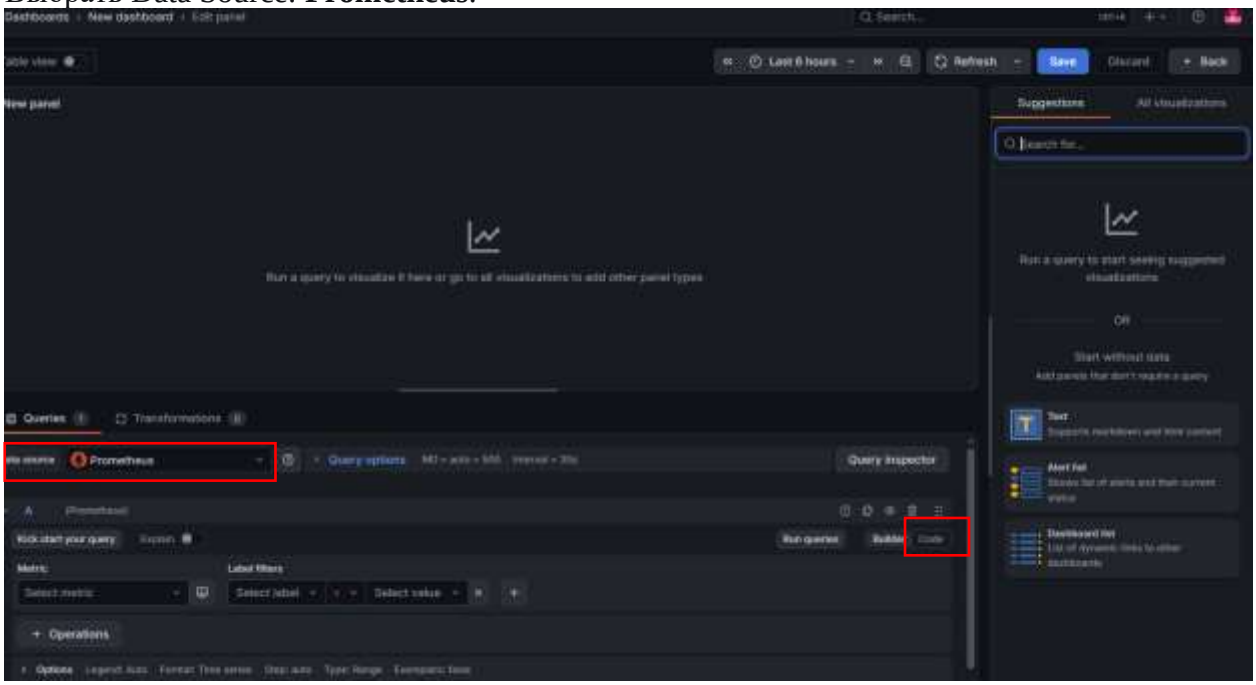
```
C:\Users\Anna>kubectl get secret -n monitoring prometheus-grafana -o jsonpath="{.data.admin-password}" |
powershell -Command "$input | %{ [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($_)) }"
FIlit3FArrGX9GJpcQtCeYJPkPdNyyiK6YO9ZW0F
```

После успешной авторизации



Шаг 4. Проверка Data Source
Перейти: **Connections → Data Sources**. Prometheus должен быть добавлен автоматически.
Проверка: нажать **Test** → должно появиться «Data source is working».
Если Prometheus не добавлен, добавить вручную:
- `URL: http://prometheus-kube-prometheus-prometheus.monitoring.svc:9090`
- `Access: Server (default)`

Шаг 5. Создание dashboard
Перейти: **Dashboards → New → New Dashboard → Add visualization**.
Выбрать Data Source: **Prometheus**.



Создать панели:

Панель 1. RPS

```
sum(rate(http_requests_total[1m]))
```

– Panel title: RPS
– Unit: reqps
– Visualization: Time series

Панель 2. HTTP 5xx Errors

```
sum(rate(http_requests_total{status=~"5.."}[1m]))
```

– Panel title: HTTP 5xx Errors
– Visualization: Time series

Панель 3. Error Rate

```
100 * sum(rate(http_requests_total{status=~"5.."}[1m]))
  / sum(rate(http_requests_total[1m]))
```

– Panel title: Error Rate
– Unit: percent (0-100)
– Порог: 1%

Панель 4. p95 Latency

```
histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le)
)
```

– Panel title: p95 Latency
– Unit: s (seconds)

Панель 5. CPU Usage

```
sum(rate(container_cpu_usage_seconds_total{container="web-app"}[1m]))
```

– Panel title: CPU Usage
– Unit: none или cores

Панель 6. Memory Usage

```
sum(container_memory_working_set_bytes{container="web-app"}) / 1024 / 1024
```

– Panel title: Memory Usage
– Unit: decmbytes (MB)

Панель 7. Pod Status

```
up{job="web-app-service"}
```

– Panel title: Pod Status
– Visualization: Stat
– Value mappings: 1 → UP (green), 0 → DOWN (red)
Сохранить dashboard под именем High-load Application.

Шаг 6. Добавление переменных (Variables)
Перейти: **Dashboard settings → Variables → Add variable**.

Переменная `namespace`
– Type: Query
– Data source: Prometheus
– Query: label_values(up, namespace)
– Refresh: On dashboard load

Переменная `pod`
– Type: Query
– Data source: Prometheus
– Query: label_values(up{namespace="$namespace"}, pod)
– Refresh: On dashboard load
Теперь в запросах можно использовать $namespace и $pod для фильтрации.
Пример:

```
sum(rate(container_cpu_usage_seconds_total{container="web-app",
pod="$pod"}[1m]))
```

Шаг 7. Настройка временного диапазона
Выбрать: Last 15 minutes – удобно для одного нагрузочного эксперимента.

Шаг 8. Запуск нагрузочного теста
В отдельном терминале:

```
k6 run test-stages.js
```

Во время теста наблюдать в Grafana:

Шаг 9. Сопоставление k6 и Grafana
Зафиксировать значения при разных уровнях нагрузки:

VU	RPS (k6)	p95 (k6)	RPS (Grafana)	p95 (Grafana)	CPU	Memory
10						
25						
50						
100						
200						

Примечание: RPS и p95 из k6 и Grafana могут не совпадать, потому что:
– k6 измеряет latency с клиента (включая сеть), Grafana/Prometheus – на сервере;
– k6 использует свои бакеты, Prometheus – свои;
– k6 считает за интервал теста, Prometheus – за [1m].

Шаг 10. Настройка Alert
Перейти: **Alerting → Alert rules → New alert rule**.
Пример: оповещение при Error Rate > 1%.
– `Query: 100 * sum(rate(http_requests_total{status=~"5.."}[1m])) / sum(rate(http_requests_total[1m]))`

– Condition: IS ABOVE 1
– Evaluation interval: 1m –
– For: 2m

Задания для самостоятельного выполнения

Задание 1. Установка и настройка Grafana

1. Проверить наличие Grafana в namespace monitoring.
2. Найти Pod и Service Grafana.
3. Выполнить port-forward.
4. Открыть Web-интерфейс и войти.
5. Проверить наличие Data Source Prometheus.

Результат:

– команды;
– вывод `kubectl get pods -n monitoring;`
– скриншот Web-интерфейса Grafana;
– скриншот проверки Data Source.

Задание 2. Создание dashboard

Создать dashboard `High-load Application` с панелями:
– RPS;
– HTTP 5xx Errors;
– Error Rate;
– p95 Latency;
– CPU Usage;
– Memory Usage;
– Pod Status.

Для каждой панели указать:
– название;
– PromQL-запрос;
– тип визуализации;
– единицу измерения.

Результат:

– скриншот dashboard;
– таблица с PromQL-запросами;
– объяснение, что показывает каждая панель.

Задание 3. Наблюдение за нагрузкой

Запустить `k6 run test-stages.js.` Во время теста наблюдать dashboard.
Определить:

1. Когда начинается рост CPU?
2. Как изменяется RPS?
3. Когда начинает расти количество ошибок?
4. Как изменяется память?
5. Есть ли момент, когда увеличение нагрузки перестаёт приводить к росту RPS?
6. Совпадает ли момент изменения внутренних метрик с ростом latency?

Результат:

– скриншот dashboard во время теста;
– таблица;
– сравнение p95 из k6 и Grafana для 100 VU: оба значения, разница в мс и %, объяснение расхождения.

Задание 4. Анализ точки деградации

Используя dashboard, определить момент начала деградации.

1. Найти нагрузку, при которой начинает расти p95.
2. Определить, как изменяется CPU.
3. Определить, как изменяется Memory.
4. Проверить изменение RPS.
5. Проверить количество ошибок.
6. Определить нагрузку, при которой начинается деградация.
7. Сформулировать гипотезу о причине.

Заполнить таблицу:

Показатель	До деградации	После деградации
VU		
RPS		
p95		
p99		
Errors		
CPU		
Memory		

Результат:

– скриншот dashboard с отмеченной точкой деградации;
– таблица;
– используемые PromQL-запросы;
– краткий анализ;
– гипотеза о причине деградации.

Задание 5. Анализ взаимосвязи метрик

Исследовать взаимосвязь между RPS, CPU, Memory, Latency, Errors.
Сформулировать вывод по схеме:

При увеличении _____ показатель _____ увеличился с _____ до _____.

Одновременно показатель _____ изменился с _____ до _____.

Это может свидетельствовать о _____.

Задание 6. Экспорт dashboard

1. Экспортировать dashboard в JSON: `Dashboard settings → JSON Model → Copy to clipboard.`
2. Сохранить JSON в файл `high-load-dashboard.json.`
3. Импортировать dashboard заново: `Dashboards → Import → Upload JSON file.`

Результат:

– файл `high-load-dashboard.json`;
– скриншот успешного импорта.

Контрольные вопросы

1. Что такое Grafana и для чего она используется?
2. Чем Grafana отличается от Prometheus?
3. Какую роль выполняет Prometheus в системе мониторинга?
4. Что такое Dashboard и Panel?
5. Для чего используется визуализация Time series?
6. Почему для анализа высоконагруженной системы важно наблюдать метрики во времени?
7. Как подключить Prometheus в качестве источника данных Grafana?
8. Что такое PromQL и где он используется в Grafana?
9. Как получить RPS с помощью PromQL?
10. Как определить количество HTTP 5xx-ошибок и их долю?
11. Почему одного графика CPU недостаточно для определения причины деградации?
12. Какие метрики необходимо сопоставлять при анализе производительности?
13. Как по графикам определить приближение системы к saturation point?
14. Почему рост нагрузки не обязательно должен приводить к пропорциональному росту RPS?
15. Чем гипотеза о bottleneck отличается от доказанного результата измерения? Как правильно сформулировать гипотезу по результатам эксперимента?